

React Native Cheat Sheet — React Native 0.87, React 19.2, TypeScript

New Architecture only · Hermes V1 · Strict TypeScript API · Expo SDK 57. Full reference: <https://reactnative.dev>

CREATE & RUN

```
# Expo (recommended)
npx create-expo-app@latest
npx expo start           # dev server
npx expo run:ios | run:android
# Without a framework
npx @react-native-community/cli init App
```

Needs Node ≥ 22.13, JDK 17+, Xcode, Android Studio. **Dev Menu:** Ctrl+M / shake / d in the terminal. **DevTools:** press j. Fast Refresh is on by default.

ARCHITECTURE

JS thread	Runs your React code on Hermes V1.
UI thread	Owens the real native views.
JSI	Synchronous C++ bridge between them; replaced the serialized bridge.
Fabric	Renderer: render → commit → mount, shadow tree laid out by Yoga.
Turbo Modules	Native modules loaded lazily via a registry.
Codegen	Generates native interfaces from TS specs. Legacy bridge removed in 0.82–0.84. useLayoutEffect, Suspense and transitions behave as on the web.

CORE COMPONENTS

View	Layout container (a div with no text).
Text	The <i>only</i> place text may appear.
Image	source={uri} or require().
TextInput	value + onChangeText.
ScrollView	Renders everything; small lists only.
FlatList	Virtualized list; the default choice.
SectionList	Grouped/sectioned virtualized list.
Pressable	Touch primitive; replaces Touchables.
Modal	Full-screen overlay.
Switch	Boolean toggle.
ActivityIndicator	Indeterminate spinner.
SafeAreaView	iOS insets (prefer safe-area-context).

No DOM: no div/span, no CSS classes, onPress not onClick.

STYLING

```
const s = StyleSheet.create({
  card: { padding: 12, borderRadius: 8 },
});
<View style={[s.card, on && s.active]} />
```

Units are density-independent points, not px. No cascade, no inheritance (except inside Text).

useColorScheme()	'light' 'dark' null
PlatformColor()	Native system colours.
StyleSheet.absoluteFill	Fill the parent.
hairlineWidth	Thinnest visible line.
useWindowDimensions()	Reactive width/height.

Alternatives: Styled Components, NativeWind, Tamagui, Unistyles.

FLEXBOX (YOGA)

flexDirection defaults to 'column', not row as on the web.

flex: 1	Fill available space.
justifyContent	Along the main axis.
alignItems	Across the cross axis.
gap / rowGap	Spacing between children.
flexWrap	'nowrap' (default) 'wrap'
position	'relative' (default) 'absolute'
aspectRatio	Width/height ratio.

Android 15+ is edge-to-edge (0.86): use useSafeAreaInsets().

LISTS

```
<FlatList
  data={places}
  keyExtractor={({p} => p.id}
  renderItem={({ item }) => <PlaceCard place={item} />
  onEndReached={fetchNextPage}
  onEndReachedThreshold={0.5}
  refreshControl={<RefreshControl refreshing={r} onRefresh={refetch} />}
/>
```

Tuning: initialNumToRender, maxToRenderPerBatch, windowSize, getItemLayout, memoized items. FlashList is the drop-in faster alternative.

TEXT INPUT & FORMS

```
<TextInput value={name} onChangeText={setName}
  placeholder="Name" keyboardType="default"
  returnKeyType="done" autoCapitalize="words" />
```

Wrap screens in KeyboardAvoidingView (behavior: padding on iOS, height on Android). Keyboard.dismiss() to close. Validation: React Hook Form + Zod.

TOUCHES & GESTURES

```
<Pressable
  onPress={onPress} onLongPress={onLong}
  hitSlop={8}
  style={({ pressed }) => [s.btn, pressed && s.btnPressed]}
  android_ripple={{ color: '#ddd' }} />
```

Responder system: onStartShouldSetResponder → onResponderMove → onResponderRelease; PanResponder wraps it. **Gesture Handler** runs gestures on the UI thread: GestureDetector + Gesture.Pan().

ANIMATIONS

```
Animated.timing(opacity, { toValue: 1,
  duration: 250, useNativeDriver: true
}).start();
```

useNativeDriver: true keeps animation off the JS thread — since 0.85 it works with layout props too.

```
// Reanimated
const x = useSharedValue(0);
const st = useAnimatedStyle(() => ({
  transform: [{ translateX: x.value } ] });
x.value = withTiming(100);
```

Also: interpolate, Animated.event, LayoutAnimation, Lottie for designer animations.

NAVIGATION

```
const RootStack = createNativeStackNavigator({
  screens: {
    Tabs: { screen: TabsNavigator },
    PlaceDetails: { screen: PlaceDetailsScreen },
  },
});
navigation.navigate('PlaceDetails', { placeId });
```

Type params with StaticParamList; read them with useRoute(). Peer deps: react-native-screens, react-native-safe-area-context.

Expo Router = file-based: app/_layout.tsx, app/place/[id].tsx, <Link href>, router.push().

PLATFORM & DEVICE

```
Platform.OS           // 'ios' | 'android'
Platform.select({ ios: a, android: b })
```

File extensions resolve automatically: x.ios.tsx, x.android.tsx, x.native.tsx, x.web.tsx.

AppState	active / background / inactive
Linking	Open URLs, handle deep links.
Share	Native share sheet.
BackHandler	Android hardware back.
PermissionsAndroid	Runtime permissions.

STATE & DATA

Local	useState / useReducer
Global	Context, Zustand, Redux Toolkit, MobX
Server	TanStack Query (cache, retry, offline)

```
const { data, fetchNextPage } =
useInfiniteQuery({
  queryKey: ['places'], queryFn: listPlaces });

const res = await fetch(url, { signal: ac.signal });
```

iOS blocks plain HTTP (ATS); Android blocks cleartext. Android emulator reaches the host at 10.0.2.2.

STORAGE & SECURITY

AsyncStorage	Async key-value, unencrypted . Never for tokens.
MMKV	Fast, synchronous, optionally encrypted.
SQLite	expo-sqlite: relational, queryable.
SecureStore	Keychain / Keystore. Tokens go here.

Never ship secrets in the bundle — anyone can read it. Use OAuth 2.0 + **PKCE** via react-native-app-auth, in the system browser. Deep links are hijackable: prefer universal/app links, never put a token in one.

Offline: NetInfo + a local write queue, replayed on reconnect.

DEBUG · TEST · PERF

React Native DevTools (press j): console, breakpoints, network, React profiler. Flipper and the remote Chrome debugger are gone.

```
jest --preset @react-native/jest-preset
render(<AddPlaceScreen />);           // RNTL
await userEvent.press(screen.getByText('Save'));
detox test | maestro test flow.yaml // E2E
```

Perf: 60 fps across two threads. Drop console.log in release, memoize list items, use the native driver, lazy-require heavy screens, profile with Peretto / Instruments.

NATIVE MODULES

```
// src/native/HapticTap.ts
import { TurboModuleRegistry } from 'react-native';
export interface Spec extends TurboModule {
  tap(style: 'light'|'medium'|'heavy'): void;
}
export default TurboModuleRegistry
  .getEnforcing<Spec>('HapticTap');
```

Spec → codegenConfig → **Codegen** → implement in Kotlin (Android) and Swift/Obj-C++ (iOS). UI components use codegenNativeComponent + a Fabric ViewManager.

Expo alternative: the **Expo Modules API** (Swift/Kotlin DSL) plus config plugins.

BUILD & RELEASE

```
eas build --profile production --platform all
eas submit --platform ios
eas update --branch production # OTA
./gradlew bundleRelease        # AAB
```

Android: upload keystore + Play App Signing, ship an AAB, tracks internal → closed → open → production. **iOS:** certificate + app id + provisioning profile → archive → TestFlight → App Store.

OTA updates ship JavaScript and assets only — never native changes. A binary only accepts updates with a matching **runtime version**.